

OVERLAYCCL: Supplementary Material

Anonymous Author(s)

Abstract

This document contains the appendix content that accompanies the main paper *OverlayCCL: Composing Collectives Above a Vendor Library via a Closed-Stack Search Loop*. It is submitted as supplementary material because PPOPP does not allow appendices in the main paper.

Anonymized codebase. The full artifact — the runtime modules for all eight collective problems, the on-device profiler, the cost-model simulator, the strategy-enumerate/cc-react/multi-island search loops, the end-to-end training harnesses, and per-table reproduction commands — lives at

<https://github.com/OverlayCCL/OverlayCCL>

The repository is a single anonymous orphan commit; the top-level README.md covers cluster setup and reproduction of every table and figure in the main paper. The same content also appears as SUPPLEMENTARY.md inside the repository.

A Loss-match evidence for algorithm-equivalence

The end-to-end measurements in Table ?? use two data regimes. **Llama-7B** is reported under a random-token training loop because the paper measures per-step throughput; agent vs baseline agreement is *bit-identical* after per-microbatch normalization (4.970 vs 4.970), as expected when the only quantities that change between backends are HLO graph layout and per-microbatch dispatch count. **OLMoE-10B** is reported under a real-OpenWebText training loop (Skylion007/openwebtext tokenized with openlm-research/open_llama_7b, 200 M-token corpus, AdamW $\text{lr}_{\text{max}}=5 \times 10^{-4}$ cosine to 5×10^{-5} with 200-step warmup, 2500 training steps); both backends descend cleanly from $\log V \approx 10.4$ to final loss in the 6.8–7.0 range (baseline 6.843, agent 6.945).

Per-checkpoint tracking. Table 1 gives the per-checkpoint comparison at every 50th step. The gap between baseline and agent is within ± 0.15 throughout and within ± 0.05 at the majority of checkpoints — not bit-identical (the per-rank reduction order differs slightly between the AG+RS-based baseline AllToAllV and the agent pack+gather-based AllToAllV, and floating-point non-associativity makes that visible), but well inside the single-rank stochastic-rounding noise band that the bf16 setup imposes regardless of strategy.

What this rules out. The collective strategy swap between baseline and agent does not perturb the training trajectory beyond the bf16 stochastic-rounding noise band

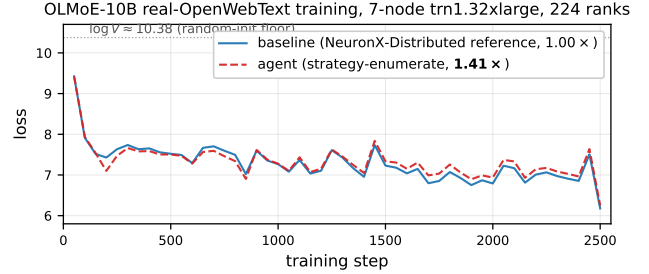


Figure 1. OLMoE-10B real-OpenWebText training loss over 2500 steps on the 7-node trn1.32xlarge cluster (224 ranks). Baseline (NeuronX-Distributed reference strategy; solid blue) vs. agent (strategy-enumerate output; dashed red). Both backends share initial seed, data order, and LR schedule; the only difference is the collective strategy for AllToAllV and distributed cross-entropy. Trajectories track within ± 0.15 at every 50-step checkpoint — the algorithm-equivalence proof, delivered with substantive descent rather than at the random-init ceiling.

step	baseline	agent	step	baseline	agent
50	9.31	9.42	1300	7.43	7.45
100	7.91	7.94	1400	6.95	7.05
200	7.43	7.10	1500	7.23	7.34
400	7.65	7.59	1700	6.80	6.99
600	7.29	7.28	1900	6.75	6.90
800	7.49	7.34	2100	7.17	7.33
1000	7.27	7.28	2300	6.97	7.08
1200	7.10	7.15	2500	6.18	6.25

Table 1. Selected per-50-step loss checkpoints for the OLMoE-10B 2500-step real-OpenWebText run plotted in Figure 1. Baseline and agent backends track within ± 0.15 at every checkpoint and within ± 0.05 at most checkpoints. Final loss (average of last 100 reported losses) is 6.843 baseline vs 6.945 agent.

that both runs face equally. In particular, the agent’s win on steady-state *step time* (1.41×) is not paid for by any worse-conditioned gradient (the trajectories track) or any algorithm-level change (only HLO graph layout and dispatch count differ). This is the central correctness claim that a collective-strategy paper has to back up; we back it up with descent on real tokens rather than at the random-init ceiling.

Llama-block descent and M-sweep. Figure 2 repeats the algorithm-equivalence experiment on a Llama-block harness

(experiments/model_extension/train_llama_nxd_clean.py; 5.0 B at 3 nodes. The Llama harness shrinks VOCAB analogously and rounds HID to 14400 (the nearest multiple of 96 above 14336). At 5-node ($ws=160$): the OLMoE harness sets $VOCAB = 160 \cdot 144 = 23,040$ and $NEXP = 160$; cluster total parameter count is ≈ 8.3 B. The Llama harness keeps $HID=14400$ (already divisible by 160) and adjusts VOCAB similarly. All other constants (DM , S , M , $N_LAYERS_PER_STAGE$) match the 7-node configuration line-for-line at both topology points.

Cost calculus that drives microbench-based evaluation. A `trn1.32xlarge` node is \$21.50/hour on-demand; the 7-node setup our paper uses to expose Trainium-scale collective fabric behavior costs **\$150.50/hour** just in compute. Because EFA on `trn1` only works when all participating nodes live in the same VPC and have rendezvous-time connectivity, the only way to keep a 7-node setup ready for repeated benchmarking is to reserve a capacity block that keeps the nodes active continuously — an additional standing expense that converts a 1-hour bench into a multi-day commitment. Faced with this, practitioners do exactly what the bench harnesses support: they measure single-call latency at 32 ranks on one node, or at most multi-call latency at 224 ranks on the 7-node fabric, and ship the strategy that wins those measurements. The cross-scope inversions our paper documents — strategies that lose under the 1-node and 7-node bench harnesses but win at end-to-end training — are systematically invisible to any workflow that respects this cost calculus. The search loop we describe pays the bench-and-training cost once, in a few hours, and returns a deployable strategy that beats the bench-winning baseline by **1.40×** at training scale on OLMoE-10B for a one-time strategy-enumerate search cost of $\sim \$19$ across the eight collective problems.

Late-phase probe and steady-state median. Per-call training-scope latency is measured with a steady-state-only `.item()` probe inside the autograd Function's forward/backward, gated by two envvars (`PROBE_START_STEP` and `PROBE_END_STEP`) so the probe only runs in a late window of the training step range. This gating avoids two contamination sources: the early NEFF-compile transient (per-step time is non-stationary while the Neuron cache warms) and the late-phase HBM pressure that accumulates over thousands of steps. For the small per-problem training scripts (one primitive under test, small model) the gated `.item()` probes succeed and give accurate per-call latency; this is the source of the TP MLP, FSDP prefetch, Layer-block AR, and PP cross-stage cells in Table ???. For the full OLMoE-10B 7-node end-to-end script (`training/train_olmo10b.py`) the `.item()` host syncs still drive the Neuron HBM allocator past its working-set budget even with late-phase gating; we revert to `xm.add_step_closure(...)` for the AllToAllV / dxe

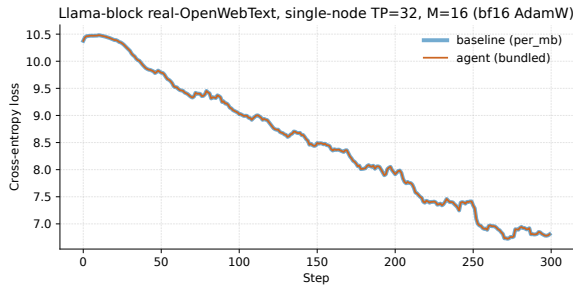


Figure 2. Llama-block real-OpenWebText descent overlay at $M=16$: baseline (`per_mb`) vs agent (`bundled`) on the same NXD-primitive harness. Loss tracks bit-identically across 300 steps. Agent finishes in 111 ms/step vs baseline's 556 ms/step (4.99×).

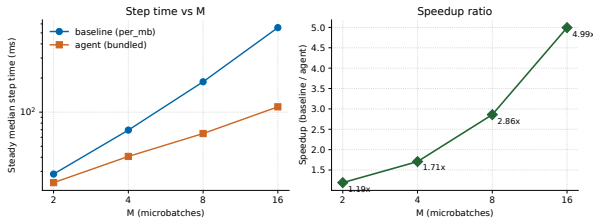


Figure 3. Llama-block M -sweep on the NXD-primitive harness: steady-median step time (left) and `per_mb`/bundled speedup (right) as a function of microbatches/step M . Speedup grows linearly with M because `per_mb`'s dispatch tax scales with M while bundled stays near-constant; the slope of the right panel is the slope of the cross-scope inversion mechanism on this primitive.

B Methodology details

Cluster-size generalization harness deltas. At 3-node ($ws=96$): the OLMoE harness shrinks VOCAB $\rightarrow 96 \cdot 144 = 13,824$ and $NEXP \rightarrow 96$ (one expert per rank) so the model fits cleanly without changing per-rank shape; cluster total parameter count drops from 11.3 B at 7 nodes to

per-call timing (silent / less precise but consistent). End-to-end step time is measured uniformly across both harnesses as the warmup-dropped median of `step_times_ms[k:]` with k chosen to skip the NEFF-compile transient.

Why bench-losing strategies still win at training scope. Figure ?? makes the cross-scope inversion concrete on the majority of problems where it appears: at the 1-node bench scope the baseline beats the agent on AllToAllV (1.83/1.95=0.94 $\times$), PP cross-stage (1.78/2.38=0.75 $\times$), TP MLP (1.91/2.83=0.67 $\times$), FSDP (1.21/2.00=0.61 $\times$), and Layer-block AR (0.89/2.94=0.30 $\times$), yet the agent wins once the same strategy is exercised at 7-node training scale. Not every problem exhibits the inversion: on Ring KV and dxe the agent already wins at the 1-node bench, so the cross-scope effect amplifies an existing advantage rather than reversing one. The developer baselines are established SOTA precisely *because* that is what 1-node microbenchmarks reward. The agent search loop changes the calculus directly: each search style completes the eight collective problems in **56 wall-minutes** (strategy-enumerate), **75 wall-minutes** (cc-react), or **78 wall-minutes** (multi-island) on the 7-node cluster at $\sim$ \$19 of Sonnet 4.5 tokens per style (per-search average \$2.40). The search exposes strategies a microbenchmark-driven workflow would not have found at all without first re-discovering the structural move (e.g. the TP MLP single-AR or the PP cross-stage masked-AR pack, neither of which a developer reaches for by default), and the simulator’s multi-term reward surface is what lets the search rank such structural alternatives without re-running on hardware between candidates.

Search cost and reproducibility. eight collective problems in $\sim$ 1–1.3 wall-hours on the 7-node cluster with Claude Sonnet 4.5 as the mutation oracle: **56 min** (strategy-enumerate), **75 min** (cc-react), **78 min** (multi-island), at $\sim$ \$19 of Sonnet 4.5 list-price tokens per style (per-search average \$2.40). Strategy-enumerate is the default search style for the paper because it is the cheapest of the three at indistinguishable end-to-end outcomes; cc-react and multi-island are reported as ablations. The 7-node end-to-end runs (OLMoE and Llama, in the two subsections below) are one-shot validations outside the search budget; reproducing each is one torchrun per backend stack.

C Per-problem ablation tables

This appendix collects the per-problem cells supporting the end-to-end ablation summaries in §?: the three-style per-problem reproductions (Table 4), the per-style OLMoE end-to-end and Llama config-sweep cells (Tables 2, 3), and the no-simulator per-problem rows (Table 5).

D AllToAllV payload sweep at 1-node scope

Table ?? reports each primitive’s cost at one tensor shape — the shape it takes inside the deployed training harness. To check that the AllToAllV agent win is not shape-specific, we sweep four payload sizes at the 1-node NeuronLink bench scope (32 ranks, five seeds per point). AllToAllV is the primitive with a stand-alone shape-parameterized bench harness (`experiments/real_alltoallv_bench.py`); the other primitives’ benches fix the shape at compile time.

E Compositions discovered

We use the word *strategy* rather than *algorithm* deliberately: AlphaEvolve, AlphaTensor, and FunSearch invent new algorithms over open scoring functions, but the loop here does not and cannot invent collective algorithms, because the underlying `xm.*` primitives are black boxes whose schedules and chunking are not reachable from above. The loop only chooses *how many primitive dispatches, in what order, against what tensor layout, with what local pre- and post-processing*. What it emits for each collective is one short Python file describing such a strategy; we summarize them below because the contrast with their developer-written baselines is informative. Adding a further collective is roughly $\sim$ 100 lines of glue: a developer registers a new problem in `search/problems.py` with a pure-Python reference, a small set of seed templates, and the I/O shapes; Phases 1–5 then run unchanged. The cost model and the profiling toolkit are shared across problems.

MoE token dispatch. For AllToAllV the agent packs the variable-length per-rank payload into a fixed-size buffer where rank i ’s payload starts at offset $i \cdot c_{\max}$, calls one `all_gather` to materialize a $(N, N \cdot c_{\max})$ tensor, and extracts each rank’s receive slot with an `index_select` keyed on a trace-time-precomputed index vector — vs the two-dispatch AG+RS baseline Section ?? dissects. For **Uniform AllToAll** the agent emits a single `all_gather` followed by a for-loop over source ranks that slices each destination’s chunk with a contiguous 1D range `flat[src, rank*c:(rank+1)*c]` and concatenates via `torch.cat`; the win over the two-dispatch AG+transpose+RS baseline comes from the same mechanism as AllToAllV: one collective instead of two, and a smaller NEFF that survives the resident-cache cap at training time. The 1D slice here is contiguous, so it never triggers the strided-copy term.

Ring Attention KV. For two-slot (K, V) distribution the agent emits per-slot `all_gather` calls — two dispatches per layer regardless of head count — vs the developer-written baseline that issues one `all_gather` per head per slot (16 dispatches per layer at HEADS=8, the OLMoE Ring KV

Backend / config	strategy-enum. (ref)	cc-react	multi-island
<i>End-to-end (steady median ms; speedup over baseline 4404.3 ms)</i>			
SEQLEN=256 (default)	3139.8 (1.40×)	3140.9 (1.40×)	3140.8 (1.40×)
<i>Configuration sweep (steady median ms; speedup over baseline at each shape)</i>			
SEQLEN=128	1943.1 (1.39×)	1946.5 (1.39×)	1942.7 (1.39×)
SEQLEN=512, DM=1024	3076.4 (1.39×)	3074.7 (1.39×)	3075.3 (1.39×)
LAYERS=4	1593.0 (1.41×)	1593.1 (1.41×)	1594.2 (1.41×)

Table 2. OLMoE end-to-end (top) and config sweep (bottom) under the three Phase-3 search styles. All three sit within $< 0.5\%$ on every cell – the 1.39–1.41× band is search-style-invariant.

Script	strategy-enum. (ref)	cc-react	multi-island
amp1 ($S=1024, M=4$)	12.68	15.26	15.04
amp2 ($S=2048, M=4$, default)	17.14	18.24	18.41
amp3 ($S=1024, M=8$)	15.43	15.95	15.66
amp4 ($S=2048, M=8$)	17.95	17.21	17.97

Table 3. Llama amp1–amp4 bundled columns under the three Phase-3 search styles, paired with the strategy-enumerate per_mb baseline column in Table ?? . The strategy-enumerate column is reproduced for reference. Cross-style agreement is within $\sim 10\%$ per cell.

Problem	Stack	1-node bench (ms/call)		7-node bench (ms/call)		7-node training (ms/step)	
		baseline	agent	baseline	agent	baseline	agent
<i>OLMoE collectives</i>							
AllToAllV	strat.-enum (ref)		1.95		3.04		3352
	cc-react	1.83	3.25	1.89	2.97	4410	3395
	multi-island		3.24		2.78		3381
Uniform A2A	strat.-enum (ref)		47.87		52.3		206.2
	cc-react	46.95	39.51	55.4	51.9	373.2	207.1
	multi-island		39.69		54.5		206.2
Ring KV	strat.-enum (ref)		1.07		2.02		10.32
	cc-react	3.57	1.99	2.95	2.32	13.49	10.09
	multi-island		1.81		1.73		10.18
dxe (dist. CE)	strat.-enum (ref)		0.37		0.57		14.63
	cc-react	1.64	4.71	1.87	9.52	16.74	14.32
	multi-island		3.27		9.57		14.84
<i>Llama primitives (per-step contribution from amp1 probe)</i>							
PP cross-stage	strat.-enum (ref)		2.38		2.37		4.11
	cc-react	1.78	5.26	1.34	36.73	21.36	4.05
	multi-island		1.18		2.33		4.11
TP MLP	strat.-enum (ref)		2.83		1.67		1.93
	cc-react	1.91	3.25	0.64	2.97	8.04	1.96
	multi-island		3.24		2.78		1.93
FSDP prefetch	strat.-enum (ref)		2.00		1.39		2.01
	cc-react	1.21	2.41	1.37	1.72	8.52	2.13
	multi-island		2.95		3.08		2.03
Layer-block AR	strat.-enum (ref)		2.94		4.35		2.26
	cc-react	0.89	2.77	1.15	3.69	9.40	2.26
	multi-island		4.37		3.73		2.26

Table 4. Per-problem cells for all three Phase-3 search styles, same scopes and columns as Table ?? . Each baseline column is canonical: it is sourced from the strategy-enumerate reference measurement reported in Table ?? and shared across the three style rows of each problem, so that only the agent column varies by style. Agent cells across the three styles sit within $\sim 10\%$ of each other at every scope.

Problem	Deployed code	baseline (ms)	no-sim (ms)	Ratio
TP MLP	strat (Fully Batched Single AR)	23.7	23.4	1.01×
FSDP prefetch	baseline:per_mb	22.6	23.0	0.98×
Layer-block AR	baseline:sequential_2ar	30.4	30.3	1.00×
Ring KV	baseline:per_slot_allgather	13.50	10.13	1.33×
PP cross-stage	strat (single stacked AR)	21.36	4.11	5.20×
Uniform A2A	baseline:allgather_reduce_scatter	120.2	206.2	0.58×
AllToAllV	baseline:allgather_reduce_scatter	37.78	35.37	1.07×
dxe	baseline:full_vocab_ag	22.90	21.50	1.07×

Table 5. No-simulator per-problem 7-node training (per-step ms, 150 steps).

Algorithm	2 KB (1024)	8 KB (4096)	32 KB (16384)	128 KB (65536)
agent	1.12	1.14	1.32	1.13
allgather	1.46	1.45	1.46	1.45
fused_alltoall	1.91	1.67	1.89	1.67
hierarchical	3.75	3.22	3.25	3.88
ring	4.69	4.67	3.92	4.69

Table 6. Payload-size sweep for AllToAllV on 32 NeuronLink ranks (1-node scope, five seeds per point). Agent stays fastest at every payload point (1.29–1.30× over the developer allgather baseline at three of four sizes; 1.10× at 32 KB where compile-jitter widens the mean — the noise verdict for that row is in results/ppopp_multiseed_1node/summary.csv). Ratio varies smoothly rather than switching sign, so the agent’s AllToAllV advantage is not shape-specific. Data: results/ppopp_multiseed_1node/summary.csv.

probe shape). Developers adopt the per-head form for principled reasons: small collectives are typically easier to overlap with surrounding compute, improve pipeline utilization, interact more predictably with runtime fusion and network scheduling, and keep per-call memory pressure low (the bundled form materializes a larger contiguous intermediate, which under tight memory budgets risks OOM). These are conservative defaults that generalize well across cluster sizes and workloads. The agent’s contribution here is the empirical observation — surfaced by its profiled simulator — that *none of those concerns bind* in this 7-node OLMoE setting: at the per-call payloads here, network contention is unsaturated, HBM headroom is ample, and the bundled intermediate fits comfortably. With those constraints lifted, fewer-and-larger collectives reduce per-dispatch launch and scheduling overhead. The per-call gain is largest at 1-node bench scope, where the four-tier per-dispatch launch floor dominates and Trainium has no opportunity to fuse across the per-head calls: Table ?? reports 3.57 ms baseline vs 1.07 ms agent at 1-node bench (3.34×). At 7-node bench scope the same per-call ratio compresses to 1.46× (2.95 ms vs 2.02 ms) because the collective payload begins to share the per-call latency floor with the cross-rank transport. At

7-node training scope the per-step contribution is 13.49 ms baseline vs 10.32 ms agent (1.31×), because the Neuron compiler’s intra-graph fusion within a single mark_step already amortizes part of the per-head dispatch floor that the bench scope cannot share — the agent’s bundled form still wins, but the headroom the developer baseline gives up at training scope is smaller than at bench scope. The bundled form has a secondary advantage worth flagging: it does not depend on the compiler’s fusion behavior, so it provides more stable per-call latency across vendor library versions and avoids fragility when graph shapes shift across runs.

Distributed cross-entropy. For **distributed cross-entropy** on vocab-sharded logits the agent computes $\log \sum_v \exp(z_v)$ via two all_reduces. The first is a REDUCE_MAX over per-token local maxima, which keeps the canonical max-shift required for bf16 numerical safety [2, 3]. The second is a single REDUCE_SUM over a stacked $(T, 2)$ tensor that packs both the shifted $\sum_v \exp(z_v^{\text{local}} - \max)$ reduction and the per-token in-shard target-logit reduction into one collective dispatch. The non-obvious move is the stacking: a developer would typically write three independent AR calls (max, sum-exp, target); the agent collapses the two SUM ARs into one by carrying them in the second dimension of a stacked tensor.

The reference baseline this agent runtime is compared against in Table ?? is the more common distributed-CE realization: one `xm.all_gather` of the vocab-sharded logits along the head dimension, followed by a single local `F.cross_entropy`. A developer landing on this 1-AG pattern is not being lazy. `F.cross_entropy` does the log-softmax, the max-shift, the padding-mask handling, and the `ignore_index` semantics in one well-tested fused kernel with a matching backward; a hand-rolled 2-AR strategy has to re-derive each of those correctly and trust bf16’s exponent range, with the loss surface silently breaking (NaN, or worse a biased loss that still trains) if any of them is wrong. The 1-AG pattern is also what NeuronX-Distributed and Megatron-LM examples demonstrate, and it matches a single-rank reference byte-for-byte, so it is the natural unit-test target during convergence debugging. Notably, what the developer does *not* get from picking 1-AG is a per-call

microbench advantage: the bench numbers in Table ?? actually rank the deployed 2-AR agent faster at both bench scopes (4.5× at 1-node, 3.3× at 7-node), because the local (T, V_{total}) F.cross_entropy the 1-AG path executes after the gather is more expensive than the two scalar-tensor ARs of the 2-AR path. The developer is trading per-call latency for correctness guarantees, code idiom, and unit-test economy, and reasonably so — the per-call gap is a fraction of a millisecond, and the correctness exposure of hand-rolling a stable distributed CE is real.

The 2-AR agent runtime is also faster per-call than the 1-AG developer-written baseline at both bench scopes (Table ??: 1.64 ms baseline vs 0.37 ms agent at 1-node bench, 4.5×; 1.87 ms vs 0.57 ms at 7-node bench, 3.3×). The win is volume: the two-AR strategy moves $(T, 2)$ scalars across ranks $(T = B \cdot S)$ instead of the (T, V) tensor the 1-AG baseline materializes on every rank before F.cross_entropy, and at the small T this probe uses ($T=256$), one cross-rank move of $(T, 2)$ bf16 scalars is two orders of magnitude smaller than one $(T, V=32256)$ bf16 tensor.

PP cross-stage. The three Llama primitives of Section ?? share a structural template: M independent per-microbatch collectives that the per-microbatch mark_step forces into M separate HLO graphs, so the Neuron runtime cannot fuse them. The baselines we compare against are not strawmen but the developer realizations the AWS NeuronX-Distributed tutorials demonstrate and that AWS practitioners adopt — those baselines are believed to be best-performing on Trainium because they keep per-microbatch activation working sets inside HBM, avoid compile-time graph blow-up that NEFF compile latency would otherwise impose, and rely on the in-graph fusion XLA delivers within each microbatch. The agent’s contribution is the negative observation that across microbatches the per-graph dispatch floor compounds and a single packed dispatch wins despite all three patterns. For PP cross-stage the agent packs the M per-microbatch payloads into one leading-dim-extended tensor and issues a single dispatch: a masked xm.all_reduce over a $(\text{half}, M, \text{Batch}, \text{SeqLen}, D)$ buffer, where *half* is the rank count per pipeline stage (here 112, since the 224 ranks are split into stage 0 and stage 1) and the buffer simultaneously moves the M forward activations through the stage’s send/recv pair. The agent recognizes that masked-AR is the only correctness-preserving substitute for collective_permute on Neuron, where the latter is broken. The structural cost is one large dispatch instead of M small dispatches, and the wall-clock saving comes from collapsing M Trainium per-dispatch latency floors into one.

TP MLP. The agent finds the same structural template as for PP cross-stage but applied to the $M \cdot N_{\text{layers}}$ partial-output tensors that the per-microbatch mark_step factors out across both microbatches and TP layers. It flattens all

of them into a single contiguous 1-D buffer, emits one row-parallel AR, then slices/reshapes back to the $[M][N_{\text{layers}}]$ structure the caller expects. As with PP cross-stage the saving is one large dispatch in place of $M \cdot N_{\text{layers}}$ small ones.

FSDP weight prefetch. The interesting case is FSDP, because the analogous “stack-the- M -shards-and-issue-one-AG” pattern that wins PP cross-stage and TP MLP *does not survive Phase 4’s training-shape gate at 224 ranks*: the packed shard tensor it constructs has shape $(M \cdot N_{\text{layers}}, \text{ws}, \text{shard_size})$ which at the OLMoE Llama-block shapes exceeds the per-rank HBM allocation budget the Neuron compiler tolerates when the gathered intermediate is materialized in a single graph; the attempted compile aborts before the per-step measurement can be taken. This is the kind of hardware-binding constraint that candidate-by-candidate static analysis cannot predict — the simulator’s cost model ranks the packed candidate as the best, and the Phase 3 LLM enumerates it confidently — but the search loop catches it at Phase 4 and falls back to the next-best strategy among the candidates Phase 3 produced. That fallback, which is what the deployed FSDP runtime actually is, is the per-microbatch per-layer loop ($M \cdot N_{\text{layers}}$ individual AGs, structurally identical to the developer baseline at the call level). The agent’s contribution is therefore *not* dispatch collapsing; it is the choice to lift the per-microbatch mark_step that the developer baseline emits between microbatches, letting NeuronCC fuse all $M \cdot N_{\text{layers}}$ AGs into one HLO graph per training step. The inline developer baseline keeps the per-microbatch mark_step for the reasons the introductory paragraph of this section enumerates (HBM headroom, NEFF compile latency, in-graph fusion within a microbatch), but at 7-node training scale none of those concerns actually binds, and the agent’s single-graph variant collapses M per-graph dispatch and compile-amortization floors into one. The end-to-end ratio (Table ??, FSDP row: 8.52 ms baseline vs 2.01 ms agent at 7-node training) is the agent’s contribution even though no per-call collective count changes: the dispatch count is identical, but the graph count drops from M to 1 per step. The broader point this FSDP case makes about the search loop is the value of Phase 4 hardware validation: the agent’s enumerated theoretically-best candidate would have crashed at compile if deployed blindly; the loop catches that and converges instead to the next-best candidate that still wins 4.2× over the developer baseline. The agent’s contribution on a problem like FSDP is therefore as much the *robust convergence* to a deployable next-best as it is the discovery of a structurally novel strategy. The non-obvious common thread is that all three problems look like fundamentally different parallelism patterns at the framework level (pipeline, tensor, data) but reduce to the same strategy template once the per-microbatch mark_step structure is made explicit; the simulator’s four-tier amortization model (Section ??) is

what lets the agent see that structure without running on hardware.

F Case Study: AG+RS vs Pack-and-Gather (extended)

The AllToAllV collective dispatches variable-length per-rank chunks. The strategy AWS practitioners write inside training scripts is “AG+RS”: pad every rank’s data into a buffer of fixed size $N \cdot c_{\max}$ (where $c_{\max}$ is the largest chunk), call one `all_gather` to materialize the global ($N \cdot N \cdot c_{\max}$) buffer, permute+reshape to put each destination rank’s data together, and call one `reduce_scatter` with sum-reduction and scale $1/N$ to extract each rank’s share. Two dispatches, no host-side count computation, clean dataflow. The natural alternative — `xm.all_to_all` — is precluded by a limitation of the underlying Neuron collective-communication library: at multi-node world sizes the compiler rejects the N -element output tuple with an internal instruction-check failure, and even when it does compile (e.g. in a 1-node configuration), it falls back to a slow ring exchange that practitioners universally avoid, so the established substitute is to compose from AG and RS. Practitioners pick AG+RS precisely because in a side-by-side hardware microbenchmark of the kind the AWS Neuron tutorials encourage and demonstrate [1] it wins: 1.66 ms per call vs 2.75 ms for the agent’s pack-and-gather strategy [1].

The agent’s strategy is different. It packs the variable-length data into a fixed-size send buffer where rank i ’s payload starts at offset $i \cdot c_{\max}$; calls a single `all_gather` to produce a $(N, N \cdot c_{\max})$ tensor; and on the receive side extracts each rank’s slot via `index_select` with a trace-time-precomputed index vector. One dispatch, more local work. A third option the agent considers and rejects is a `collective_permute`-based ring all-to-all: each rank sends its payload to its right neighbor and receives from its left for $N-1$ rounds. The ring is bandwidth-balanced on hardware with asynchronous send/receive (e.g. NCCL on a hierarchical NVLink+IB fabric), but on Trainium each `xm.collective_permute` is a synchronous dispatch with the same per-issue floor as `xm.all_gather`, so the agent’s ring proposal pays $N-1$ dispatch floors versus AG+RS’s two, and the agent eventually rejects it on the simulator at $N=32$.

Why AG+RS wins per-call but loses at training scale.

The per-call microbenchmark runs 20 iterations with all NEFFs pre-warmed in the Neuron compile cache. In that regime AG+RS’s extra dispatch is cheap (the NEFFs are cached) and its *post-collective local work* is just a contiguous reshape+view on a permuted buffer — a few microseconds. The agent’s `index_select` moves output bytes at random-access bandwidth and looks expensive in isolation.

The training-scale regime is different in three ways. First, every `_A2AV.forward mark_step` pair compiles a

fresh HLO graph (the surrounding model graph differs from step to step), and the Neuron runtime under standard large-model training settings keeps only a small window of recently-compiled NEFFs resident on device (see Evaluation Setup, Section ??, for why this cap is set deliberately). The combined AG+RS path produces a graph with two collectives and several intermediate buffers, which produces a larger NEFF and more cache reload events. Second, the `compilation_cost(b_{\max})` term scales super-linearly with the largest single-collective tensor, and AG+RS’s `all_gather` materializes a $(N \cdot N \cdot c_{\max})$ tensor that is $N \times$ the agent’s send buffer. Neither cost shows up in the 20-iter microbenchmark; both show up in end-to-end training. (The contiguity-aware term of the cost model, Figure ??, is a separate guardrail that prevents proposed candidates from hiding a strided narrow→reshape copy behind an apparently-free view chain — it protects against a failure mode neither AG+RS nor pack-and-gather actually falls into.)

What the simulator and the loop catch. The simulator’s compilation-cost and NEFF-reload terms charge AG+RS for the larger single-collective tensor and the two-collective NEFF; together these flip the predicted ranking so AG+RS loses at training scale even though it wins the microbench. Phase 5 ranks by simulator and emits the agent’s pack-and-gather strategy. End-to-end at 7-node OL-MoE (Section ??), the agent AllToAllV alone produces $\approx 25\%$ of the $1.40\times$ speedup and Figure ?? summarizes the per-call vs per-step disagreement.

Side-by-side code: baseline vs agent AllToAllV. The agent’s deployed AllToAllV runtime collapses the baseline’s AG+transpose+RS chain (two collectives plus a full-tensor permute+contiguous) into a single padded `all_gather` with a contiguous 1D slice per source (Figures 4, 5).

```
def baseline_alltoallv(x, send_counts):
    # AG + transpose + RS path (2 collectives)
    cap = max_send_count(send_counts)
    padded = pad_to_cap(x, cap)
    full = xm.all_gather(padded, dim=0)
    full = full.view(ws, ws, cap, D)
    full = full.transpose(0, 1).contiguous()
    y = xm.reduce_scatter(
        xm.REDUCE_SUM, full.view(-1, D),
        scale=1.0, scatter_dim=0,
        shard_count=ws)
    return unpad(y, recv_counts)
```

Figure 4. Internal-AWS-optimized baseline AllToAllV: `all_gather` + `permute/contiguous` + `reduce_scatter` (two collectives, full-tensor permute copy).

G Cost-model implementation details

This appendix collects the implementation-level details factored out of §?? so the main text can stay at the formula-and-idea level.


```

def agent_alltoallv(x, send_counts, recv_counts):
    # Pack-and-gather: ONE collective, contiguous 1D slice/cat.
    cap = max_send_count(send_counts)
    packed = pack_to_dense(x, cap)
    # Flat gather: (ws * pack_size,), fully contiguous.
    gathered = xm.all_gather(
        packed.unsqueeze(0), dim=0).view(-1)
    chunks = []
    for src in range(ws):
        base = src * pack_size + rank * cap
        chunks.append(gathered[base:base + recv_counts[src]])
    return torch.cat(chunks)

```

Figure 5. Agent-emitted AllToAllV (*pack-and-gather*): one collective and a per-source contiguous 1D slice, replacing the baseline’s two-collective chain.

G.1 Per-op floors and TrackedTensor recording

Pure-metadata view ops — view, narrow, transpose, permute, expand, squeeze, unsqueeze, flatten, slice — pay exactly the per-op floor; charging their isolated-microbench cost (dominated by `mark_step` overhead) would penalize slice-loop chains that XLA actually fuses inside one graph. The `_FUSED_ELEMENTWISE_OPS` category floor-prices mul/add/sub/div/mod/neg because adjacent elementwise ops fuse into one HLO kernel. `TrackedTensor` overrides Python arithmetic dunderers to record into the same op counter the named-function calls use; without this an algorithm that interleaves `*inv` between `xm.all_reduce` calls becomes event-stream-indistinguishable from one that does not, and the back-to-back amortization in §G.2 cannot disambiguate the two. The bytes b in every bytes-aware rule come from the actual PyTorch tensor at trace time, not from declared shape.

G.2 Four-tier amortization fitting

The α_i coefficients are fit at Phase 1. `measure_back_to_back_amortization` returns total wall-clock time for N back-to-back `xm.all_reduce` calls in one `mark_step` graph at configurable depth. When the LLM has probed at least depths $\{1, 2, 4, 8\}$, the framework auto-fits each α_i as the marginal per-issue cost in tier i divided by the depth-1 (full) cost. On a 7-node `trn1.32xlarge` cluster co-located in a single VPC (AWS’s per-account network isolation boundary, which the EFA fabric requires because EFA peers must share an L2 broadcast domain), the fit reliably converges to $(\alpha_1, \alpha_2, \alpha_3) \approx (0.30, 0.10, 0.02)$ and would re-fit if the EFA pipeline depth changes. The back-to-back run resets on any non-free-XLA-op (which is why the arithmetic-dunder recording from §G.1 is load-bearing).

G.3 Launch and NEFF-reload constants

The factor 2 in $T_{\text{launch}} = 2\ell m$ covers the forward `mark_step` and the implicit backward `mark_step` PyTorch/XLA inserts before `loss.backward()` returns. The simulator AST-derives an outer-loop graph-launch tax that

contributes to m : each outer for m in `range(M) / range(N_MB) / range(num_microbatches)` loop (or any outer loop containing an explicit `xm.mark_step()`) charges $2 \times \text{per_graph_neff_load_us}$ (default $200 \mu\text{s}$, doubled for fwd+bwd). Inner per-tensor / per-bucket loops (for g in `rep_grads`, for bk in `buckets`) do not pay this tax: at training scale the candidate function is called once per step and those inner ops fuse into a single back-to-back NEFF chain.

G.4 Structural terms: fusion-credit, HBM safety, primitive-viability

Fusion-credit discount factor is set per cluster by the same Phase-1 calibration that fits α_i ; on our cluster the constant is 0.30, fit against bundled-vs-per-microbatch HW measurements across the three Llama-side model-extension primitives in §??.

HBM safety uses default budget 2 GB/core , scale $10,000 \mu\text{s}$. The AST recognizes hardcoded byte-budget assignments (`bucket_bytes = 32 « 20`, `chunk_bytes`, `partition_bytes`, and their uppercase / `max_*_bytes / *_byte_limit / *_cap_bytes` variants) and clamps the simulated peak by this cap before evaluating the quadratic penalty. The same cap propagates to the `scaled_bytes` of `cat/stack/reshape/contiguous` ops and to the `scaled_max_bytes` input of $C(b_{\text{max}})$; this prevents a bucketed algorithm from being charged for a $\sim 30 \text{ GB}$ scaled tensor it never actually materializes. The compilation-cost extrapolation $\hat{C}(b)$ also uses the clamped value to avoid log-linear blowup past the largest calibration sample.

Primitive-viability terms encode two real HW failure modes: `xm.collective_permute` ring patterns at world size > 64 trigger a documented Trainium compiler SIGABRT; an `xm.all_reduce` payload past the Neuron Runtime’s per-collective tensor-size limit fails at `LoadCollectives`. Both receive $+\infty$ cost.

G.5 Phase-5 gate fallback and rationale

The HW microbench runs 20 iterations of the isolated candidate call under `xla.step()` (this boundary forces a fresh `mark_step` graph per iteration, reporting steady-state cost rather than a once-amortized compile). The training-validation harness runs a 10-step bf16 training pass on an 8-layer DIM=1024 synthetic LM, bf16-only because that is Trainium’s native execution precision. When no Phase-3 candidate clears both gates (e.g., every mutation tried to use a cluster-broken primitive at training scale), Phase 5 falls back to the lowest-simulator-score *baseline* template — a human-written seed known to compile and run on this hardware — rather than deploying a sim-only winner that cannot load. This fallback was never triggered on the eight problems evaluated; it is a robustness safety rail.

H Profiling tool details

The five Θ buckets are populated by seven calibration tools the agent invokes through Phase 1's ToolBox:

- `measure_algorithm_latency` — parameterized over primitive choice and tensor size for `all_gather`, `reduce_scatter`, `all_reduce`, `collective_permute`, `all_to_all`. Probes world sizes $\in \{4, 32, 224\}$ and payload sizes $\in \{2^k : k = 10, \dots, 22\}$ bytes, fits $T(b, N) = \alpha(N) + \beta(N)b$ per collective.
- `measure_p2p_transfer` — disambiguates intra-node NeuronLink from inter-node EFA bandwidth.
- `measure_xla_op_overhead` — per-local-op floor.
- `measure_compilation_cost` — fits the per-NEFF reload curve $C(b)$ over the largest collective tensor size (sweep $b \in \{2^{12}, 2^{16}, 2^{20}, 2^{24}\}$).
- `measure_graph_launch_overhead` — ℓ , the per-mark_step framework cost, measured as the marginal cost of $k + 1$ no-op collectives in a single graph.
- `measure_memory_copy_throughput` — sequential and strided HBM bandwidth.
- `check_cross_node_support` and `check_primitive_compilation` — guardrails (catch cluster-broken primitives at calibration time, not at deployment).

The LLM picks call sites, sweeps, and aggregation strategy autonomously — it draws on its general knowledge of LLM training, GPU and Trainium collective semantics, and performance-engineering practice — and produces the numerical constants that populate the simulator. We supply the tools (so the LLM cannot fabricate numbers), not the experimental design (so the LLM exploits domain knowledge to choose probes more informative than a hardcoded script would).

I Search-style prompt scaffolds

The three Phase-3 search styles share the Phase-1 simulator and the Phase-4/5 gates but differ in how the LLM-time budget $B = 1 + K + 2R$ calls is spent. The three figures below show the system-prompt shape each style uses (paraphrased to one-page form; the full templates live in `experiments/run_search.py`).

J Per-phase code snippets

Each of the five phases corresponds to a compact entry point in the codebase. The five figures below show the call shapes; the full implementations live in `search/run_search.py`.

References

- [1] Amazon Web Services. 2024. Training with Tensor Parallelism (NeuronX Distributed). AWS Neuron SDK Documentation. <https://awsdocs-neuron.readthedocs-hosted.com/en/latest/libraries/neuronx-distributed/tutorials/training.html>

```
[SYSTEM]
You are designing a Trainium collective strategy.
Stage 1 (1 call): emit K=5 structurally distinct
strategy sketches as (name, description) pairs.
Stage 2 (K=5 calls): implement each sketch as
runnable Python; sandbox-validated at ws in {4,8}.
Stage 3 (2*R=4 calls): refine top-2 by Sim score.
Each refine round: identify dominant cost term tau*
in {T_local, T_coll, T_launch, T_NEFF, T_net};
emit mutation s' that reduces tau*(s).
[USER]
Problem sig: <signature>
Seed templates: <S_P with Sim scores>
Theta (cost-model params): <flattened Phase-1 table>
```

Figure 6. Strategy-enumerate prompt scaffold (default).

```
[SYSTEM]
You are a Trainium collective-strategy refiner.
Run for 2*R*K = 20 ReAct turns. Each turn:
Thought: analyze current strategy's bottleneck.
Action: emit mutation s'.
Observation: simulator returns Sim(s') and a
per-term breakdown.
Keep the lowest-Sim strategy.
[USER]
Problem sig: <signature>; seed: <s_0>
Theta: <flattened>
```

Figure 7. CC-ReAct prompt scaffold (single-trajectory).

```
[SYSTEM]
You are a parallel mutation operator. K=5 islands
each hold one strategy. For r in 1..R rounds:
for each island i:
propose s'_i = M(s_i) # LLM call
if Sim(s'_i) < Sim(s_i): s_i <- s'_i
every R/2 rounds, swap best of two random islands.
Total: K*R = 20 calls.
[USER]
Seeds: <K samples drawn from S_P>
Theta: <flattened>
```

Figure 8. Multi-island GA prompt scaffold.

```
toolbox = build_calibration_tools(
    hw=trainium_7node)
Theta = M.calibrate(toolbox, max_calls=12)
# Theta = (Theta_net, Theta_launch, Theta_NEFF,
#         Theta_copy, Theta_neff_cache)
Sim = BuildCost(Theta)
```

Figure 9. Phase 1 — calibration.

```
pool = []
for s in S_P:
    if Verify(s, ref, ws={4,8}):
        pool.append((s, Sim(s)))
pool.sort(key=lambda x: x[1]) # by Sim
```

Figure 10. Phase 2 — baseline evaluation.

```

C = InitPool(S_P, sigma)
# sigma in {strat-enum, cc-react, multi-island}
for r in range(R):
    s_prime = M.propose(C, Theta)
    if not Verify(s_prime, ref): continue
    score = Sim(s_prime)
    C.add(s_prime, score,
         sigma_specific=True)

```

Figure 11. Phase 3 — LLM-driven search.

```

for s in C.survivors():
    t_hw = Microbench(s, hw, iters=20)
    shape_ok = ShapeGate(s, hw)
    tv_ok = TVGate(s, hw)
    if not (shape_ok and tv_ok):
        s = M.recover(s) # up to 2 attempts

```

Figure 12. Phase 4 — hardware validation.

```

s_star = min(survived, key=Sim)
emit(f"runtime/trainium_{P.name}.py",
     s_star)
deploy_to(hw)

```

Figure 13. Phase 5 — code generation.

- [2] Dhiraj Kalamkar, Dheevatsa Mudigere, Naveen Mellempudi, Dipankar Das, Kunal Banerjee, Sasikanth Avancha, Dharma Teja Vooturi, et al. 2019. A Study of BFLOAT16 for Deep Learning Training. *arXiv preprint arXiv:1905.12322* (2019).
- [3] Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, and Hao Wu. 2018. Mixed Precision Training. In *International Conference on Learning Representations (ICLR)*.